

Modelos de Deep Learning

Attention en Seq2Seq

Universidad ORT Uruguay

17de Noviembre, 2025

Nos basaremos fuertemente en el paper [Bahdanau et al., 2014].

Extracto del abstract:

*The models proposed recently for neural machine translation often belong to a family of **encoder-decoders** and encode a source sentence into a **fixed-length vector** from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to **automatically soft-search** for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly.*

Encoder-Decoder: modelo clásico

En el Encoder-Decoder setting, el encoder lee la secuencia input

$$\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_{T_x})$$

y devuelve un **context vector** $\mathbf{c}$.

El enfoque más común en 2014 era usar una RNN

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}) \quad \text{y} \quad \mathbf{c} = q(\{\mathbf{h}_1, \dots, \mathbf{h}_{T_x}\}).$$

Por ejemplo, previamente se solía usar f una LSTM y $q(\{\mathbf{h}_1, \dots, \mathbf{h}_T\}) = \mathbf{h}_T$.

Decoder y modelado de la traducción

El decoder es entrenado para predecir la siguiente palabra $\mathbf{y}_t$ dado un context vector $\mathbf{c}$ y todas las palabras predichas previamente $\{\mathbf{y}_1, \dots, \mathbf{y}_{t-1}\}$:

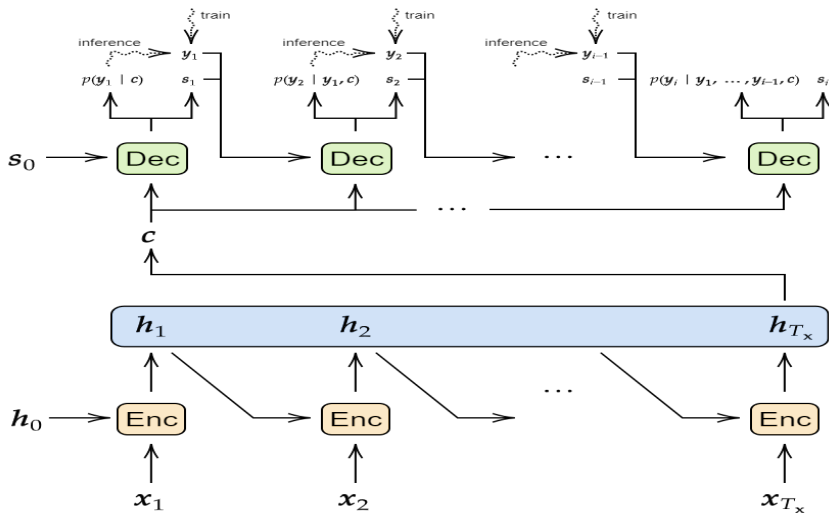
$$p(\mathbf{y}) = \prod_{t=1}^T p(\mathbf{y}_t \mid \{\mathbf{y}_1, \dots, \mathbf{y}_{t-1}\}, \mathbf{c})$$

Con una RNN como decoder:

$$p(\mathbf{y}_t \mid \{\mathbf{y}_1, \dots, \mathbf{y}_{t-1}\}, \mathbf{c}) = g(\mathbf{y}_{t-1}, \mathbf{s}_t, \mathbf{c})$$

donde g devuelve la probabilidad de $\mathbf{y}_t$, y $\mathbf{s}_t$ es el hidden state del decoder.

Traducción antes de [Bahdanau et al., 2014].



RNN machine translation – Learning to align

Cita de [Bahdanau et al., 2014]:

*[...] we introduce an extension to the **encoder-decoder model** which **learns to align** and **translate** jointly. Each time the proposed model **generates a word** in a translation, it **(soft-)searches** for a set of positions in a **source sentence** where the most relevant information is concentrated. The model then **predicts a target word** based on the **context vectors** associated with these **source positions** and **all the previous generated target words**.*

Modelo con atención (Bahdanau et al.)

En el nuevo modelo:

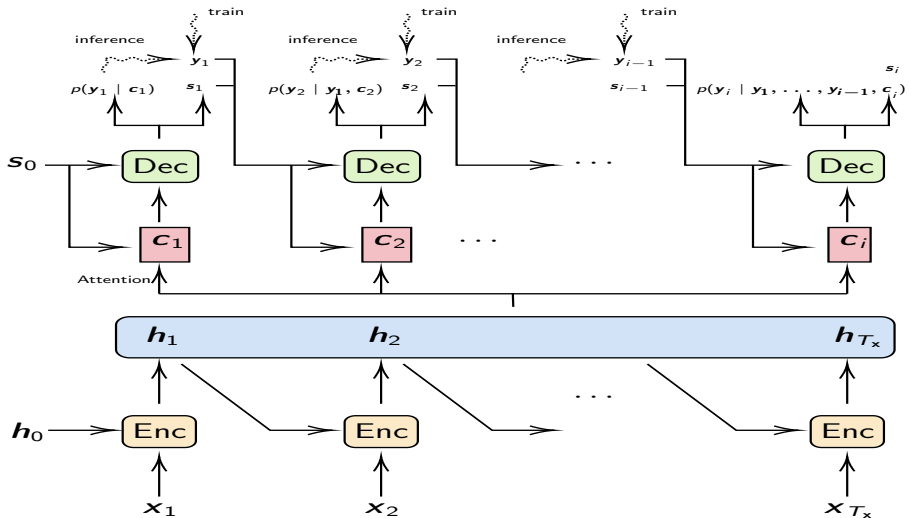
$$p(\mathbf{y}_i \mid \mathbf{y}_1, \dots, \mathbf{y}_{i-1}, \mathbf{x}) = g(\mathbf{y}_{i-1}, \mathbf{s}_i, \mathbf{c}_i)$$

El hidden del decoder es

$$\mathbf{s}_i = f(\mathbf{s}_{i-1}, \mathbf{y}_{i-1}, \mathbf{c}_i).$$

Aquí el context vector **depende de i** : un $\mathbf{c}_i$ distinto para cada palabra target.

Arquitectura propuesta en [Bahdanau et al., 2014].



Cálculo del context vector

El context vector $\mathbf{c}_i$ depende de la secuencia de hiddens del encoder:

$$\mathbf{c}_i = \sum_{j=1}^{T_x} \alpha_{ij} \mathbf{h}_j.$$

Los pesos:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})} = \text{Softmax}_j[e_{i1}, \dots, e_{iT_x}].$$

Con

$$e_{ij} = a(\mathbf{s}_{i-1}, \mathbf{h}_j),$$

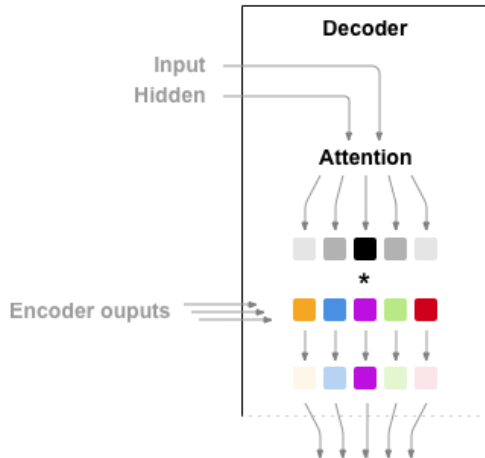
un *score de alineación* input-output.

Comparación con terminología (Keys, Values, Queries)

- Keys: hiddens del **encoder** $\{\mathbf{h}_1, \dots, \mathbf{h}_{T_x}\}$;
- Values: (combinaciones) hiddens del **encoder** $\{\mathbf{h}_1, \dots, \mathbf{h}_{T_x}\}$;
- Queries: hiddens del **decoder** $\{\mathbf{s}_1, \dots, \mathbf{s}_{T_y}\}$.

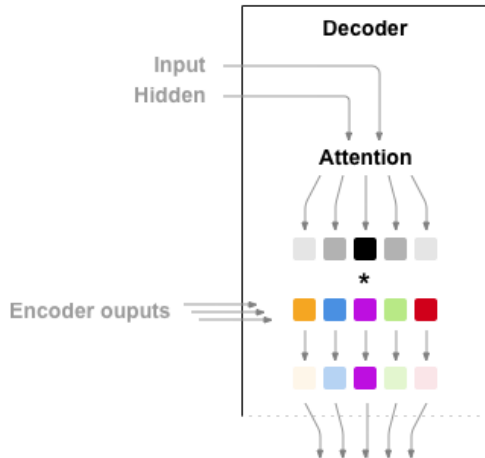
Resumiendo

[...] We can understand the approach of taking a **weighted sum** of all the annotations (**hiddens**) as computing an expected annotation, where the expectation is over possible alignments. Let α_{ij} be a probability that the target word $\mathbf{y}_i$ is aligned to, or translated from, a source word $\mathbf{x}_j$. Then, the i -th context vector $\mathbf{c}_i$ is the expected annotation over all the annotations with probabilities α_{ij} . [...]



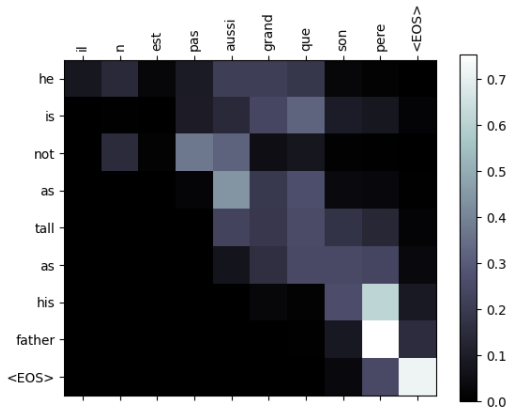
Resumiendo

[...] The probability α_{ij} , or its associated energy e_{ij} , reflects the importance of the annotation $\mathbf{h}_j$ with respect to the previous hidden state $\mathbf{s}_{i-1}$ in deciding the next state $\mathbf{s}_i$ and generating $\mathbf{y}_i$. Intuitively, this implements a mechanism of **attention in the decoder**. The decoder decides parts of the source sentence to pay attention to. [...]



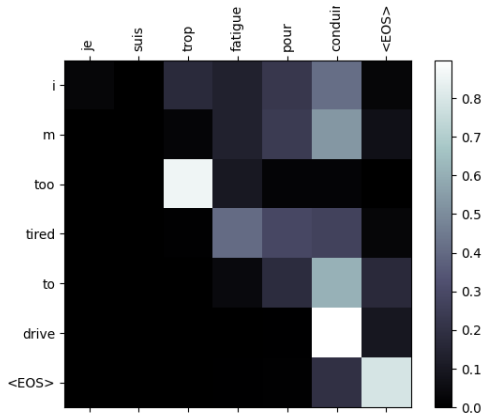
Visualizando la atención

'il n est pas aussi grand que son pere'



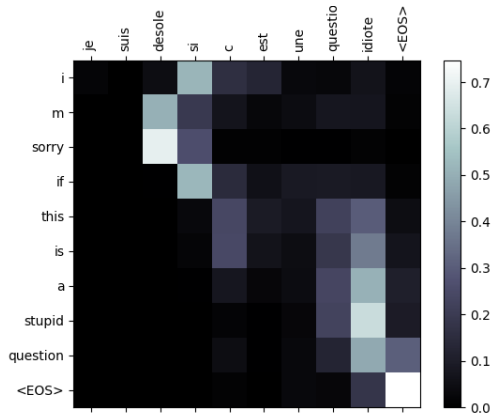
Visualizando la atención

'je suis trop fatigue pour conduire'



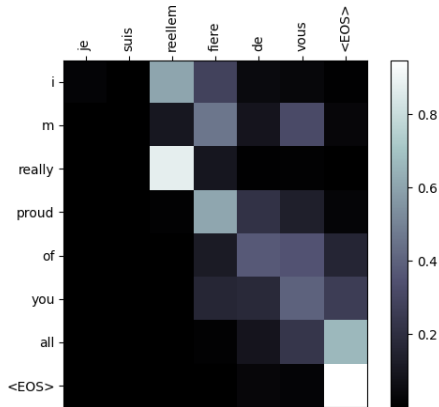
Visualizando la atención

'je suis desole si c est une question idiote'



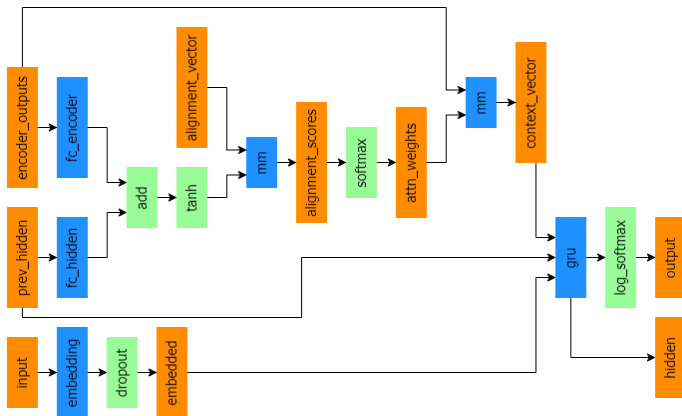
Visualizando la atención

'je suis reellement fiere de vous'



Arquitectura concreta propuesta en el paper

We parametrize the alignment model a as a *feedforward neural network* which is *jointly trained* with all the other components of the proposed system. [...]



Atención de Bahdanau: explicación basada en [Robertson, 2017]

En cada paso del decoder, el modelo **decide a qué partes de la secuencia fuente prestar atención** para generar la siguiente palabra.

- El encoder produce hiddens $\mathbf{h}_1, \dots, \mathbf{h}_{T_x}$.
- El decoder tiene un hidden previo $\mathbf{s}_{i-1}$ y la palabra anterior $\mathbf{y}_{i-1}$.
- La atención calcula pesos α_{ij} sobre los $\mathbf{h}_j$ y un context vector $\mathbf{c}_i$.
- El decoder usa $\mathbf{c}_i$ para actualizar su estado y predecir $\mathbf{y}_i$.

Entradas del decoder en el paso i

En el paso i del decoder:

- **Input**: la palabra target anterior $\mathbf{y}_{i-1}$.
- **prev_hidden**: hidden state previo del decoder $\mathbf{s}_{i-1}$.
- **encoder_outputs**: hiddens del encoder $\{\mathbf{h}_1, \dots, \mathbf{h}_{T_x}\}$.

La palabra anterior se encaja:

$$\text{embedded} = E(\mathbf{y}_{i-1}), \quad \tilde{\mathbf{e}}_{i-1} = \text{Dropout}(E(\mathbf{y}_{i-1})).$$

Embedding y context vector se usarán luego como entrada al GRU del decoder.

Qué tan relevante es cada $\mathbf{h}_j$ para producir $\mathbf{y}_i$ dado $\mathbf{s}_{i-1}$

Primero se proyectan el hidden del decoder y los hiddens del encoder:

$$W_s \mathbf{s}_{i-1} \quad \text{y} \quad W_h \mathbf{h}_j.$$

Para cada posición j se calcula:

$$\text{sum}_{ij} = W_s \mathbf{s}_{i-1} + W_h \mathbf{h}_j,$$

y se aplica una no linealidad:

$$\tilde{\mathbf{v}}_{ij} = \tanh(\text{sum}_{ij}).$$

Estos $\tilde{\mathbf{v}}_{ij}$ son vectores intermedios de **alineación**.

Cálculo de atención: scores y softmax

A partir de los vectores $\tilde{\mathbf{v}}_{ij}$, se obtienen **scores escalares** de alineación:

$$e_{ij} = \mathbf{v}^\top \tilde{\mathbf{v}}_{ij}, \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}$$

lo que da un score para cada posición j del input.

- $\tilde{\mathbf{v}}_j \rightarrow$ vector intermedio dependiente de j ,
- $\mathbf{v}$ (no depende de i y j) $\rightarrow$ parámetros del mecanismo de atención.
- α_{ij} es la **probabilidad** de alinear $\mathbf{y}_i$ con la posición j del input.
- El vector $\boldsymbol{\alpha}_i = (\alpha_{i1}, \dots, \alpha_{iT_x})$ son los **pesos de atención**.

Cálculo del context vector

Pesos de atención α_{ij} y hiddens del encoder $\mathbf{h}_j$ forman el **context vector**:

$$\mathbf{c}_i = \sum_{j=1}^{T_x} \alpha_{ij} \mathbf{h}_j.$$

Interpretación probabilística:

- α_{ij} puede verse como la probabilidad de que $\mathbf{y}_i$ esté alineada con $\mathbf{x}_j$.
- $\mathbf{c}_i$ es la **esperanza** de las anotaciones $\mathbf{h}_j$ bajo esa distribución.
- El modelo decide “dónde mirar” en la secuencia fuente para generar $\mathbf{y}_i$.

Decoder GRU con atención

El decoder combina tres fuentes de información:

- embedding de $\mathbf{y}_{i-1}$: $\tilde{\mathbf{e}}_{i-1}$,
- hidden anterior del decoder: $\mathbf{s}_{i-1}$,
- context vector actual: $\mathbf{c}_i$.

El GRU del decoder actualiza su estado:

$$\mathbf{s}_i = \text{GRU}([\tilde{\mathbf{e}}_{i-1}, \mathbf{c}_i], \mathbf{s}_{i-1}).$$

El nuevo hidden $\mathbf{s}_i$ está influenciado tanto por la historia del decoder como por las partes relevantes de la secuencia fuente seleccionadas por la atención.

Salida: distribución sobre la próxima palabra

A partir del hidden $\mathbf{s}_i$ se genera la distribución sobre la siguiente palabra:

$$\mathbf{o}_i = W_o \mathbf{s}_i + \mathbf{b}_o,$$

$$p(\mathbf{y}_i \mid \mathbf{y}_{1:i-1}, \mathbf{x}) = \text{Softmax}(\mathbf{o}_i).$$

Resumiendo el paso i :

1. Calcular scores e_{ij} y pesos α_{ij} .
2. Formar el context vector $\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j$.
3. Actualizar el hidden del decoder $\mathbf{s}_i$ usando $\tilde{\mathbf{e}}_{i-1}$ y $\mathbf{c}_i$.
4. Aplicar una capa lineal + softmax para obtener $p(\mathbf{y}_i \mid \cdot)$.

Bibliografía

-  Bahdanau, D., Cho, K., and Bengio, Y. (2014).
Neural machine translation by jointly learning to align and translate.
arXiv preprint arXiv:1409.0473.
-  Robertson, S. (2017).
NLP From Scratch: Translation with a Sequence to Sequence Network and Attention.
PyTorch Tutorials.
Last Updated: Oct 21 2024.